| ordering | persistence_id | sequence_number |
|---|---|---|
| 3 | scontainer-type-key\|11 | 1 |
| 2 | scontainer-type-key\|9 | 2 |
| 5 | scontainer-type-key\|9 | 3 |
| 6 | scontainer-type-key\|11 | 4 |
| 7 | scontainer-type-key\|9 | 5 |
| 8 | scontainer-type-key\|9 | 6 |
| 9 | scontainer-type-key\|9 | 7 |

For the connection to the database, you can use `ScalikeJdbc` as you did for the projections in chapter 10. In that chapter, the specific implementation of the connection pool was `HikariCP`. This works analogously to SLF4J, where you must choose the logging implementation. `HikariCP` is to `ScalikeJdbc` what `logback-classic` is to SLF4J. In this example, use the `commons-dbcp2` included in the `ScalikeJdbc` library (http://mng.bz/o1Ny). To do so, set the JDBC and connection pool settings in your application.properties file:

```
# JDBC settings
db.containers.driver="org.postgresql.Driver"
db.containers.url="jdbc:postgresql://localhost:5432/containers"
db.containers.user="containers"
db.containers.password="containers"
# Connection Pool settings
db.containers.poolFactoryName="commons-dbcp2"
```